

CSE46901: Introduction to Robotics

Simulated Milestone 1

Professor Jaeyeon Lee
TA: Minhyuk Park

Introduction

Welcome to CSE46901. We have prepared a few milestones before the final project, in which you will use a physical robot to execute a mission given by us. The following milestones will give you an idea how to interact with [ROBOTIS' Turtlebot3 waffle Pi](#) with a [Manipulator Arm](#) using the Robot Operating System.

For more information on ROS itself, this free book from the robot's manufacturer will help you be familiar with the Robot Operating System.

Official: <https://www.robotis.us/ros-robot-programming-book-digital-copy/>

Mirror: https://ai.unist.ac.kr/~chiu/robot/books/ROS_Robot_Programming_EN.pdf

Alternatively, following wiki's instructions will be helpful to you. Start from "Navigating the ROS Filesystem" after installing ROS based on the instructions we will provide you below.

<http://wiki.ros.org/ROS/Tutorials>

Submission requirement

In accordance with the online classes only policy for March, we have changed the first milestone to be done in simulation. In this simulated milestone, you will demonstrate your ability to follow basic instructions in setting up a programming environment for ROS.

We are not trying to differentiate students' performance with milestones. The only grade that will be given is Pass/Fail. However, we do expect you to provide us with evidence that you have followed through the milestones by posting a video of each group following through with the tasks in Part 1 and 2. Requirements of what to be included in the video will be provided below.

Once your group is done with the video demonstration, please submit it on the blackboard. Each group will submit one video. However, each member will be asked to take parts in the video in specified ways to prevent free-riding.

Once we verify that you have submitted the milestone requirement, you will be awarded Pass/Fail. You can resubmit videos to challenge your Fail status until the final project time period so that all groups can eventually pass the milestone requirement with no penalty for resubmission. (i.e. no difference between teams who get it right the first try vs teams who retries). However, if we resume offline lectures, there would be logistical limit of each teams being able to physically interact with the turtlebot.

Instruction

Part1

We need you to prepare Ubuntu 20 PC for course milestones and the final project. From now onwards it will be called 'Remote PC'. It will provide you with a simulated environment and necessary visualization tools you need to interface with both the simulated and the physical robot. It is preferable for that PC to be portable, because you might want to use it for future milestones with a physical turtlebot, should we begin physical classes in the future. However, it is not necessary. We expect PCs of around 6th generation Intel i5 CPU with integrated graphics to be sufficient for this course.

The following instructions are based on <https://emanual.robotis.com/docs/en/platform/turtlebot3/quick-start/#pc-setup> Above manual might help you if you are not comfortable with our provided instruction.

We are using the ROS 1 version Noetic Ninjemys. Please select Noetic in the above website.

If we are to change to offline classes in the future, we will provide you with an Intel NUC portable computer with programs already installed along with physical turtlebot3. If you are using the provided NUC as the 'Remote PC', all programs you need are already installed.

Preparing ROS environment for the 'Remote PC'

1) Install Ubuntu 20.04.2.0 LTS (Focal Fossa, 64bit, Desktop)

- The installation file is `ubuntu-20.04.2.0-desktop-amd64.iso`
- hostname: `remote-pc`
- username: `YOUR_USER_NAME`
- password: `YOUR_PASSWORD`

2) Install some software on remote-pc:

```
sudo apt -y install vim
sudo apt -y install net-tools
sudo apt -y install openssh-server
```

```
sudo apt -y install curl
```

- You may want to remap the shortcut keys of Copy and Paste of your terminal.

3) Setup Network:

- Run ifconfig to see the IP of remote-pc. Remember the IP as IP_OF_REMOTE_PC.

4) Update Ubuntu software:

```
sudo apt-get update  
sudo apt-get upgrade
```

5) Install ROS 1 Noetic Ninjemys (full version):

- Please refer to <http://wiki.ros.org/noetic/Installation/Ubuntu>

```
sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" >  
/etc/apt/sources.list.d/ros-latest.list'
```

```
sudo apt-key adv --keyserver 'hkp://keyserver.ubuntu.com:80' --recv-key  
C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654
```

```
curl -sSL  
'http://keyserver.ubuntu.com/pks/lookup?op=get&search=0xC1CF6E31E6BADE8868B172B  
4F42ED6FBAB17C654' | sudo apt-key add -
```

```
sudo apt update
```

```
sudo apt install ros-noetic-desktop-full
```

```
echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
```

```
source ~/.bashrc
```

```
sudo apt install python3-rosdep python3-rosinstall python3-rosinstall-generator  
python3-wstool build-essential
```

```
sudo rosdep init
```

```
rosdep update
```

6) Install TurtleBot 3 ROS 1 Packages:

- Please refer to

<https://emanual.robotis.com/docs/en/platform/turtlebot3/quick-start/#pc-setup>

```
sudo apt update
sudo apt upgrade
wget
```

https://raw.githubusercontent.com/ROBOTIS-GIT/robotis_tools/master/install_ros_noetic.sh

```
chmod 755 ./install_ros_noetic.sh
bash ./install_ros_noetic.sh
```

- Press enter to continue the installation

```
sudo apt-get install ros-noetic-joy ros-noetic-teleop-twist-joy \
ros-noetic-teleop-twist-keyboard ros-noetic-laser-proc \
ros-noetic-rgbd-launch ros-noetic-rosserial-arduino \
ros-noetic-rosserial-python ros-noetic-rosserial-client \
ros-noetic-rosserial-msgs ros-noetic-amcl ros-noetic-map-server \
ros-noetic-move-base ros-noetic-urdf ros-noetic-xacro \
ros-noetic-compressed-image-transport ros-noetic-rqt* ros-noetic-rviz \
ros-noetic-gmapping ros-noetic-navigation ros-noetic-interactive-markers
```

```
sudo apt update
sudo apt upgrade
sudo apt autoremove
```

7) Create workspace and download turtlebot3 packages

```
sudo apt install ros-noetic-dynamixel-sdk
sudo apt install ros-noetic-turtlebot3-msgs
sudo apt install ros-noetic-turtlebot3
```

- At this point, you can login as a different (non-root) user if you want to avoid using the account with sudo power.

- If ~/catkin_ws does not exist, run the following commands:

```
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws
catkin_make
```

8) Set ROS_MASTER_URI and ROS_HOSTNAME in .bashrc:

- Use "ifconfig" to find the IP address of the remote pc
- Make sure the following is added to ~/.bashrc:

```
source /opt/ros/noetic/setup.bash
source ~/catkin_ws/devel/setup.bash
export ROS_MASTER_URI=http://IP_OF_REMOTE_PC:11311
export ROS_HOSTNAME=IP_OF_REMOTE_PC
```

- Source .bashrc

```
source ~/.bashrc
```

9) Download turtlebot3 package:

```
cd ~/catkin_ws/src/
git clone https://github.com/ROBOTIS-GIT/turtlebot3_msgs.git
git clone https://github.com/ROBOTIS-GIT/turtlebot3.git
git clone -b noetic-devel https://github.com/ROBOTIS-GIT/turtlebot3_simulations.git
cd ~/catkin_ws && catkin_make
```

10) Open ~/.bashrc and add the following to the end:

```
export TURTLEBOT3_MODEL=waffle_pi
```

- Then source ~/.bashrc

```
source ~/.bashrc
```

Part2

Next we need you to demonstrate your setup with teleoperation and running SLAM and Navigation code.

Running SLAM node

How to run SLAM on provided NUC

This manual is based on the following manual. You need to follow this if you are doing this on your own pc:

<https://emanual.robotis.com/docs/en/platform/turtlebot3/slam/>

This manual assumes you have completed manual on setting up your remote PC.

1. Run roscore from remote PC

```
roscore
```

2. Connect to turtlebot Pi via ssh and launch bringup

```
roslaunch turtlebot3_bringup turtlebot3_robot.launch
```

3. On remote PC launch SLAM node. This will launch SLAM node with GMapping method as default.

```
roslaunch turtlebot3_slam turtlebot3_slam.launch
```

-You can also try different slam method called Karto.

```
roslaunch turtlebot3_slam turtlebot3_slam.launch slam_methods:=karto
```

4. Run teleoperation node

```
roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch
```

-Now pilot the robot remotely to draw complete map of the field. You might need to tune parameters to get good quality map. Having a good map is prerequisite for next step, running navigation node.

Tuning guide

You can tune Gmapping parameters with following command

```
roscd turtlebot3_slam
```

```
cd config
vi gmapping_params.yaml
```

Here are parameters defined for Gmapping.

1. `maxUrange`: This parameter is set the maximum usable range of the lidar sensor.
2. `map_update_interval`: This parameter defines time between updating the map. The smaller the value, the more frequent the map is updated. However, setting this too small will require more processing power for the map calculation. Set this parameter depending on the map environment.
3. `minimumScore`: This parameter sets the minimum score value that determines the success or failure of the sensor  scan data matching test. This can reduce errors in the expected position of the robot in a large area.
4. `linearUpdate`: When the robot translates longer distance than this value, it will run the scan process.
5. `angularUpdate`: When the robot rotates more than this value, it will run the scan process. It is recommended to set this value less than `linearUpdate`.

Refer to This link for more information on parameters:

<https://emanual.robotis.com/docs/en/platform/turtlebot3/slam/#run-slam-node>

Saving map

1. Launch `map_saver` node to create map files generated from running SLAM node.
 - location is in `/home/(username)`
 - two files, `map.pgm` and `map.yaml` will be generated

Now close all nodes and move on to running navigation node

Running navigation node

Just like with a real turtlebot, we can run a navigation node on a simulated turtlebot.

0. Launch nodes in separate terminals

```
roslaunch turtlebot3_gazebo turtlebot3_world.launch
roslaunch turtlebot3_navigation turtlebot3_navigation.launch
map_file:=$HOME/map.yaml
```

1. We need to initialize AMCL parameters to tell turtlebot where it is on the map.

Click the 2D Pose estimate button in the RViz GUI menu.

Click on the map where the actual robot is located and drag the large green arrow toward the direction where the robot is facing.

Repeat step 1 and 2 until the LDS sensor data is overlaid on the saved map.

2. Launch keyboard teleoperation node on a new terminal to precisely locate the robot on the map.

```
roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch
```

3. Move the robot back and forth a bit to collect the surrounding environment information and narrow down the estimated location of the TurtleBot3 on the map which is displayed with tiny green arrows.

4. Once sufficient accuracy is attained, shut down teleoperation node with `ctrl + c`

5. Click on the 2D Nav Goal button in the RViz menu GUI.

6. Click on the map to set the destination of the robot and drag the green arrow toward the direction where the robot will be facing.

-This green arrow is a marker that can specify the destination of the robot.

-The root of the arrow is x, y coordinate of the destination, and the angle is determined by the orientation of the arrow.

-As soon as x, y, targets are set, TurtleBot3 will start moving to the destination immediately.

7. Recall from before that you can Tune navigation parameters by modifying the appropriate yaml file. Use following command to find folder for turtlebot3_navigation:

```
roscd turtlebot3_navigation
```

We will leave you this URL again as a reference for complete set of parameters :

<https://emanual.robotis.com/docs/en/platform/turtlebot3/navigation/#navigation>

Demo requirement

You will have to submit a video showing that you

- (1) can display a simulated world on Gazebo
- (2) can use teleoperation node to move the turtlebot in the simulated environment
- (3) have successfully run a SLAM algorithm to map a simulated environment in Gazebo
- (4) can use a navigation node to move the Turtlebot from one location to another in Gazebo.

After a sequence of demos showing four things, to discourage freeriding, each member should include a clip in the video with their own voice explaining one of the following. (i.e. member a will explain 1, member b will explain 2... if your team member is below or more than 5, you can skip or re-explain answered topics)

Please explain the following topics in 30 seconds or less. It does not have to be precise. You can google the topics and respond in your own words.

1. What is ROS?
2. Why do we edit bashrc in Part 1?
3. Why should we source bashrc in Part 1?
4. What is the goal of SLAM?
5. What are a few example scenarios that lidar distance sensors can give inaccurate readings?